

0) `pipeline()` আসলে কী করে?

তুমি যখন এটা লিখো—

```
from transformers import pipeline
```

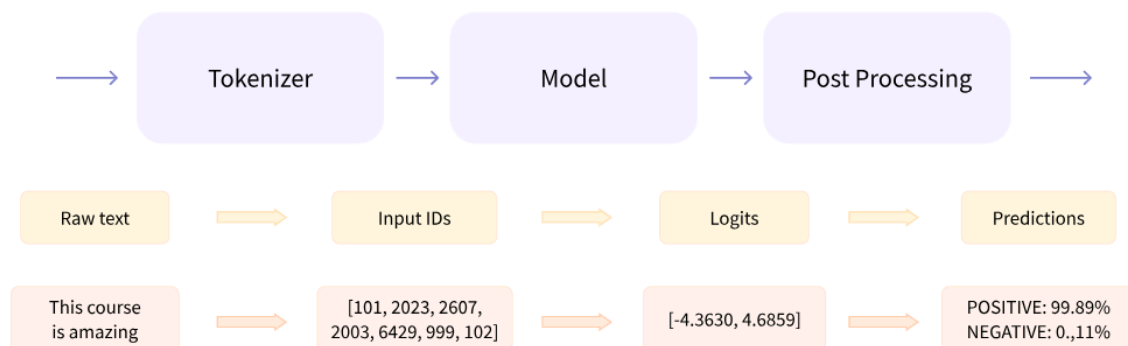
```
classifier = pipeline("sentiment-analysis")
classifier([
    "I've been waiting for a HuggingFace course my whole life.",
    "I hate this so much!",
])
```

`pipeline()` তোমার হয়ে ৩টা কাজ একসাথে করে দেয়:

1. **Preprocessing** → টেক্সটকে মডেলের ইনপুট রূপান্তর (Tokenizer)
2. **Model inference** → টেক্সট ইনপুট মডেলের ভেতর দিয়ে চালানো
3. **Postprocessing** → লগিটসকে probability + label এ বদলানো

এজন্যই আউটপুট এমন হয়:

```
[{'label': 'POSITIVE', 'score': 0.9598...},  
 {'label': 'NEGATIVE', 'score': 0.9994...}]
```



এখন এই ৩টা ধাপ খুলে দেখি।

1) Step-1: Preprocessing (Tokenizer) — টেক্সট → নাম্বার

Transformer মডেল **raw text** বুঝে না। তাকে দিতে হয় **numbers** (IDs) + কিছু helper তথ্য।

1.1 Tokenizer কী কী করে?

Tokenizer সাধারণত এসব করে:

- টেক্সটকে **token** এ ভাঙে (word/subword/punctuation)
- প্রতিটা token কে একটা **integer ID** দেয়
- কিছু **special tokens** যোগ করে (মডেল-ডিপেন্ডেন্ট)
- **padding/truncation** করে (একাধিক বাক্য একসাথে দিলে)
- **attention_mask** বানায় (কোনটা আসল, কোনটা padding)

1.2 সঠিক tokenizer কেন জরুরি?

একটা মডেল যেভাবে pretrain হয়েছে, একই **tokenizer-r** নিয়ম ফলো না করলে মডেল ভুল ইনপুট পায়। তাই আমরা checkpoint থেকে tokenizer লোড করি:

```
from transformers import AutoTokenizer
```

```
checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"  
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

1.3 Tokenize করে ইনপুট বানানো (Tensor আকারে)

```
raw_inputs = [  
    "I've been waiting for a HuggingFace course my whole life.",  
    "I hate this so much!",  
]  
  
inputs = tokenizer(  
    raw_inputs,  
    padding=True,  
    truncation=True,  
    return_tensors="pt" # PyTorch tensors  
)  
  
print(inputs)
```

এতে সাধারণত এমন ডিকশনারি আসে:

- **input_ids**: token ID গুলো
- **attention_mask**: কোন ID “real token”, কোনটা “padding”

input_ids এ কী থাকে?

ধরো এমন আসে:

- `[101, ..., 102]` → এখানে **101** ও **102** অনেক সময় special token:
 - `101 ≈ [CLS]` (শুরু বোঝায়)
 - `102 ≈ [SEP]` (শেষ বোঝায়)
- `0` → padding token (যখন বাক্য ছোট, তখন শেষে 0 বসে length equal করার জন্য)

attention_mask কেন লাগে?

Batch-এ সব বাক্যকে একই length করতে padding লাগে। কিন্তু padding যেন “meaning” হিসেবে ধরা না পড়ে, তাই:

- real token হলে mask = 1
- padding হলে mask = 0

মানে মডেলকে বলা হয়: “এই জায়গাগুলো ignore করো।”

2) Step-2: Model inference — মডেল ইনপুট নিয়ে কী আউটপুট দেয়?

2.1 **AutoModel** বনাম **AutoModelForSequenceClassification**

অনেকে এখানে কনফিউজ হয়:

(A) **AutoModel**

```
from transformers import AutoModel
model = AutoModel.from_pretrained(checkpoint)
outputs = model(**inputs)
```

এটা শুধু **base Transformer**। এটা আউটপুট দেয়:

- `last_hidden_state` (hidden states / features)

Shape সাধারণত হয়:

```
print(outputs.last_hidden_state.shape)
# torch.Size([2, 16, 768])
```

এটা বুঝা এভাবে:

- **2** = batch size (২টা বাক্য)
- **16** = sequence length (token সংখ্যা; padding সহ)
- **768** = hidden size (প্রতি token-এর জন্য 768-dim feature vector)

এই `last_hidden_state` খুব “rich representation”, কিন্তু সরাসরি **positive/negative** বলে না।

2.2 Sentiment-এর জন্য দরকার “Classification Head”

Sentiment analysis হলো **sequence classification task**। তাই আমরা ব্যবহার করি:

```
from transformers import AutoModelForSequenceClassification
```

```
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
outputs = model(**inputs)
```

এখানে base transformer-এর শেষে একটা **classification head** বসানো থাকে (সাধারণত 1-2টা linear layer)।

এখন logits-এর shape হবে:

```
print(outputs.logits.shape)
# torch.Size([2, 2])
```

কারণ:

- 2টা input sentence
- 2টা label (NEGATIVE, POSITIVE)

3) Step-3: Postprocessing — logits → probability → label

3.1 Logits কী?

```
print(outputs.logits)
# tensor([[ -1.5607,  1.6123],
#         [ 4.1692, -3.3464]])
```

এগুলো **probability** না। এগুলো “raw scores” (unnormalized)।

Training-এ সাধারণত loss function (cross entropy) logits নিয়েই কাজ করে, তাই Transformers logits-ই দেয়।

3.2 Softmax দিয়ে **probability** বানানো

```
import torch
```

```
predictions = torch.nn.functional.softmax(outputs.logits, dim=-1)
print(predictions)
# tensor([[0.0402, 0.9598],
#         [0.9995, 0.0005]])
```

এখন প্রতিটা row-এর যোগফল = 1
মানে এগুলো probability-like score.

3.3 কোন **index** কোন **label**?

মডেলের config এ mapping থাকে:

```
model.config.id2label
# {0: 'NEGATIVE', 1: 'POSITIVE'}
```

তাহলে:

- প্রথম বাক্য: NEGATIVE 0.0402, POSITIVE 0.9598 → **POSITIVE**
- দ্বিতীয় বাক্য: NEGATIVE 0.9995, POSITIVE 0.0005 → **NEGATIVE**

4) তাহলে **pipeline()** কেন এত সুন্দর আউটপুট দেয়?

কারণ **pipeline("sentiment-analysis")** ভিতরে ভিতরে প্রায় এগুলো করে:

1. checkpoint অনুযায়ী **tokenizer** লোড
2. task অনুযায়ী ঠিক model class লোড
→ sentiment হলে **AutoModelForSequenceClassification**
3. tokenize করে **input_ids**, **attention_mask** বানায়
4. model inference করে logits বের করে
5. softmax চালিয়ে probability বের করে
6. **id2label** দিয়ে label নাম বসিয়ে দেয়
7. শেষ আউটপুট বানায়: **[{label:..., score:...}, ...]**

5) একদম “**pipeline**-এর মতো” ফুল মিনিমাল রিপ্রডিউসেবল কোড

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
```

```
checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
```

```
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
```

```
texts = [
    "I've been waiting for a HuggingFace course my whole life.",
    "I hate this so much!",
]
```

```
inputs = tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
outputs = model(**inputs)
```

```
probs = torch.nn.functional.softmax(outputs.logits, dim=-1)
```

```
id2label = model.config.id2label
```

```
for i, p in enumerate(probs):
    neg = p[0].item()
    pos = p[1].item()
    pred_id = int(torch.argmax(p).item())
    print(texts[i])
    print("NEGATIVE:", neg, "POSITIVE:", pos)
    print("PRED:", id2label[pred_id])
    print()
```